CDTA SHIMMER
HUMAN ACTIVITY RECOGNITION

Classical Machine Learning | Research Evaluation Update | 15 October 2026

Technical report with additional visualizations
Scope: dataset processing, baseline features, KNN/SVM/Decision Tree, MCC and balanced accuracy, candidate recording boundaries, group-aware testing, exploratory permutation p-values, and multiclass ROC-AUC.

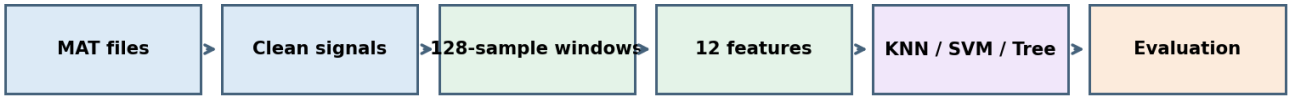


Figure: End-to-end pipeline.

Executive summary

The project processes Shimmer accelerometer recordings into 128-sample windows and 12 statistical features per window. The updated code supports repeated stratified holdout experiments, MCC- or balanced-accuracy-based hyperparameter tuning, multiclass one-vs-rest ROC-AUC, timestamp-reset candidate groups, group-held-out evaluation, and exploratory permutation testing. The current report audits implementation and illustrates methodology; it does not claim that the updated experiments have been executed on raw recordings.

Evidence boundary: The provided files contain source code and a README, but no raw MAT recordings, executed evaluation CSVs, or verified acquisition log. All schematic figures are explicitly illustrative. The class distribution is a previously reported baseline and may differ after rerunning the updated pipeline.

1. Dataset and feature engineering

The loader expects 11 numeric columns in MAT recordings: timestamp, accelerometer X/Y/Z, gyroscope X/Y/Z and four additional channels. Only accelerometer X/Y/Z are used for the 12-feature classical ML baseline. Invalid acceleration rows are removed; repeated sensor readings are reported rather than automatically deleted.

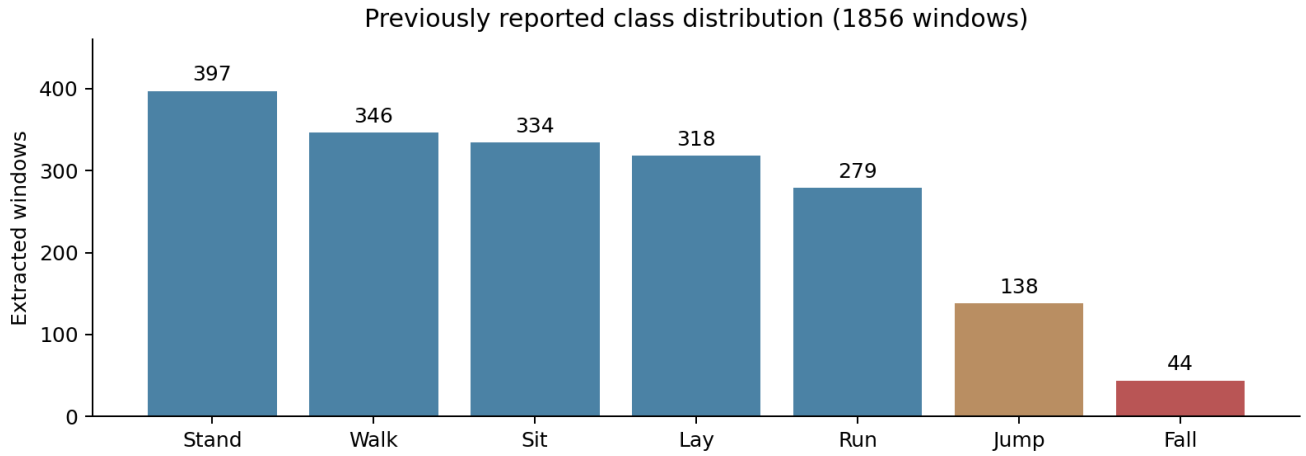


Figure: Historical class distribution supplied in project context.

Earlier baseline counts: Stand 397, Walk 346, Sit 334, Lay 318, Run 279, Jump 138, Fall 44 (total 1,856). This demonstrates class imbalance; the rare Fall class makes balanced accuracy, macro metrics, and MCC particularly important.

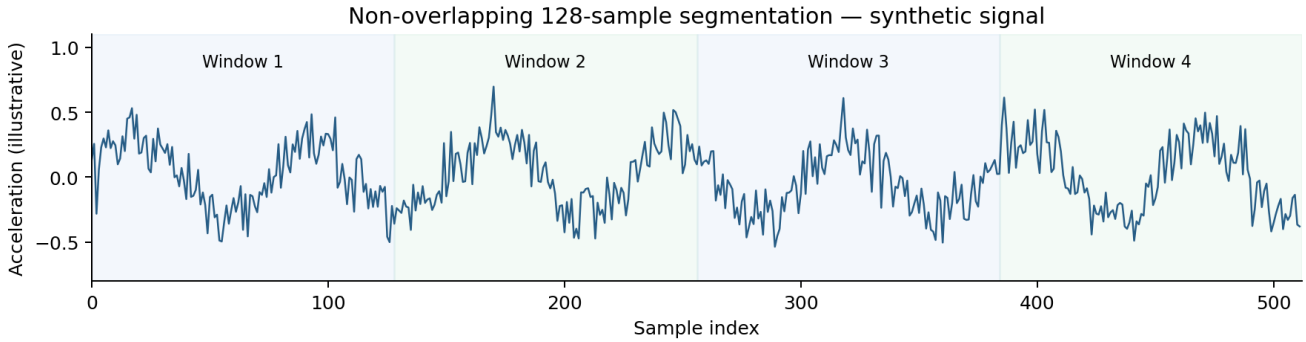


Figure: Window construction (synthetic signal).

Axis	Features per window
X	x_max, x_min, x_mean, x_std
Y	y_max, y_min, y_mean, y_std
Z	z_max, z_min, z_mean, z_std

Window duration is 128 / sampling frequency seconds. It is 2.56 s only if sampling frequency is 50 Hz; timestamp units and actual acquisition frequency require confirmation.

2. Baseline models and nested tuning

Three pipelines are compared: KNN, SVM, and Decision Tree. Median imputation is included within all pipelines; KNN and SVM also use StandardScaler. The outer repeated stratified split reserves a test partition; GridSearchCV tunes hyperparameters on the training partition. This prevents test-set preprocessing and tuning leakage, but does not remove recording-level dependence.

Model	Search dimensions
KNN	Neighbors 3, 5, 7, 9, 11; uniform/distance weights; Manhattan/Euclidean distance
SVM	RBF or linear kernels; C and gamma grid
Decision Tree	Criterion, maximum depth, minimum split and leaf sizes

3. MCC and balanced accuracy

Balanced accuracy averages recall equally over classes. Matthews correlation coefficient (MCC) summarizes association between predicted and true labels and is useful under imbalance. The updated code allows either metric to drive GridSearchCV while reporting both after evaluation. To compare tuning objectives fairly, hold the outer splits, seed, feature table, and parameter grids fixed.

Run condition	Tuning objective	Report
A	MCC	MCC, balanced accuracy, accuracy, macro F1, ROC-AUC
B	Balanced accuracy	Same evaluation metrics; compare paired run-level differences

The code may produce different optimal hyperparameters for conditions A and B. Neither metric is inherently guaranteed to outperform the other across every class.

4. Recording boundaries and data leakage

The updated MAT loader derives candidate recording IDs from non-increasing timestamp differences. This is a diagnostic heuristic, not verified evidence of new participants or independent sessions. The candidate IDs must be checked against acquisition notes, sensor restart logs, and per-class counts.

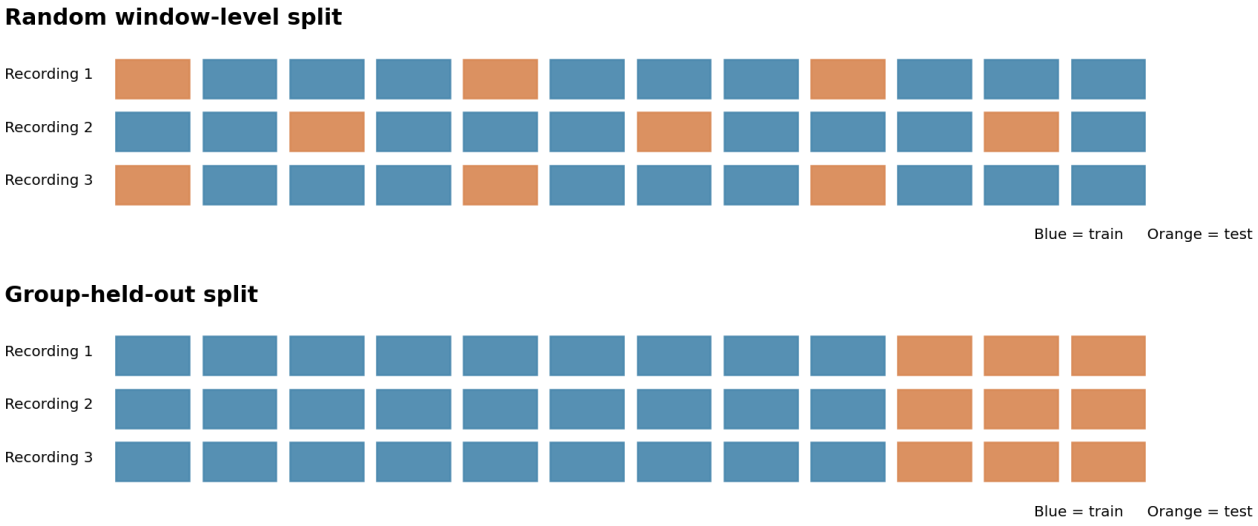


Figure: Illustration of window-level versus group-held-out splitting.

With random window-level splitting, adjacent windows from one acquisition can appear in both training and testing. With group-held-out evaluation, candidate groups are separated. This may reveal performance inflation from shared recording conditions. However, timestamp-reset groups could still be correlated; therefore describe this as candidate-group generalization, not participant-independent generalization.

5. Group-aware evaluation protocol

The updated research_evaluation.py uses StratifiedGroupKFold for the outer evaluation and group-aware cross-validation during hyperparameter search. It requires sufficient candidate groups per class and checks that every outer fold contains all classes. A five-fold design requires at least five candidate groups per class, but feasibility also depends on group sizes and class allocation.

Before accepting results: verify that each extracted window belongs to a single candidate recording. In the current design, groups are attached using each window start sample; if a window crosses a timestamp reset, its group assignment could be misleading. The window extractor should also split at verified recording boundaries, not only sample-index gaps.

6. Permutation test and p-value

The exploratory procedure shuffles training labels at the candidate-group level, fits a clone of the tuned estimator, and evaluates it on the unchanged held-out fold. For B permutations, p = (1 + number of null scores at least as high as the observed score) / (B + 1). With B = 99, the smallest attainable p-value is 0.01.

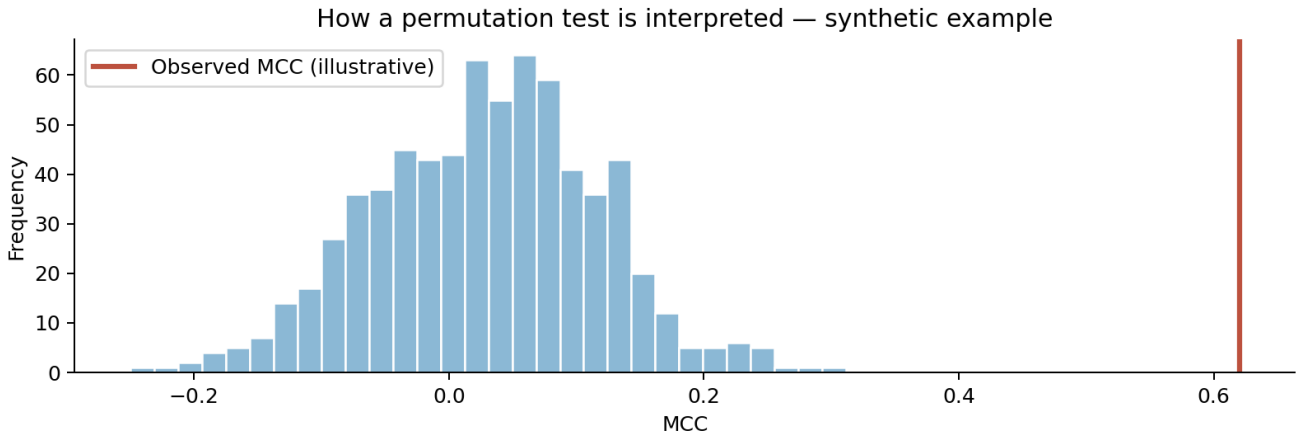


Figure: Permutation-test intuition (synthetic values).

Interpretation limitation: Hyperparameters were selected using true labels before the permutations. The resulting per-fold p-values are conditional, not a fully nested permutation test. Fold results are dependent and their p-values should not be naively combined. A rigorous significance claim requires a pre-specified permutation design that respects the actual exchangeability structure.

7. Multiclass ROC-AUC

The updated experiment calculates one-vs-rest macro ROC-AUC from predicted class probabilities. SVM uses probability estimation. This measures ranking across thresholds rather than classification quality at one fixed decision threshold. ROC-AUC can be undefined if a held-out fold lacks a class; the code records a missing value in such cases.

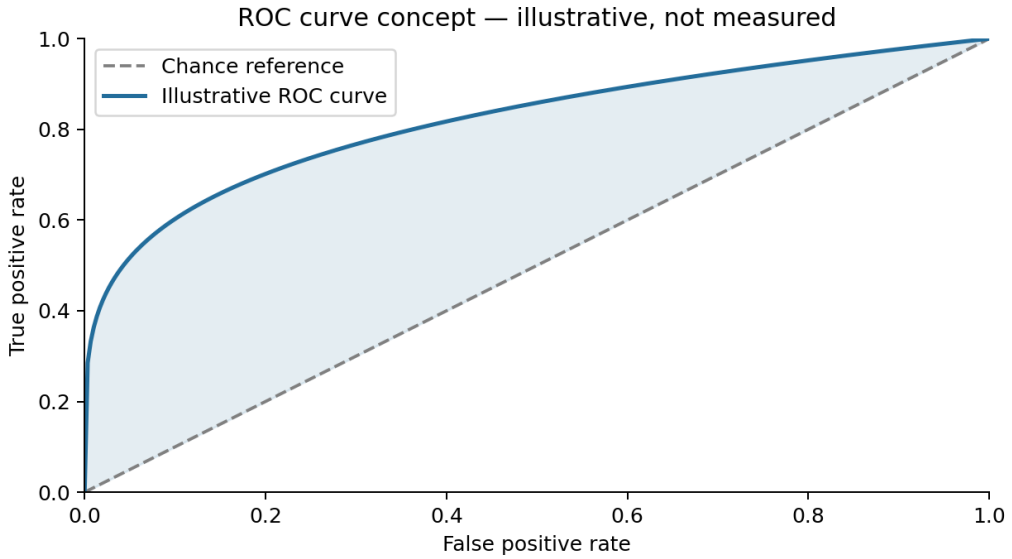


Figure: ROC-AUC interpretation (illustrative).

Because Fall is rare, also inspect Fall recall, false alarms, and precision-recall behavior. Macro ROC-AUC should complement, not replace, MCC and balanced accuracy.

8. Recommended result visualizations after execution

Plot	What to show	Reason
Candidate groups by class	Number of groups and windows per activity	Assess feasibility and imbalance
MCC / balanced accuracy comparison	Mean ± SD across 10 matched outer splits	Compare tuning objectives
Random vs grouped evaluation	Side-by-side metrics for each model	Assess sensitivity to recording dependence
ROC curves	One-vs-rest curve for each activity	Inspect threshold-independent ranking
Permutation null distribution	Observed score and null histogram	Explain exploratory p-value
Confusion matrix	Per-class errors, especially Fall vs Jump	Diagnose failure modes

9. Result recording template

Protocol	Model	Tuning	MCC mean ± SD	Balanced acc.	Macro ROC-AUC	p-value
Random window	KNN / SVM / Tree	MCC	Pending	Pending	Pending	Not run
Random window	KNN / SVM / Tree	Balanced acc.	Pending	Pending	Pending	Not run
Candidate-group held-out	KNN / SVM / Tree	MCC	Pending	Pending	Pending	Pending
Candidate-group held-out	KNN / SVM / Tree	Balanced acc.	Pending	Pending	Pending	Pending

Pending means no updated empirical run results were supplied. Do not substitute historical model scores into this table, because evaluation settings and tuning grids have changed.

10. Reproducibility and next steps

1. Confirm timestamp units and candidate boundaries with acquisition documentation. 2. Inspect group counts and ensure windows never span a reset. 3. Run 10 matched random-split experiments under both tuning metrics. 4. Run grouped evaluation only when groups are defensible and folds feasible. 5. Start exploratory permutations at B=9 for a smoke test, then B=99 or more if computationally practical. 6. Export run-level and fold-level CSVs, model settings, and plots; update this report with actual measured values.

Appendix A. Code provenance

This report is based on the supplied updated package CDTA_updated.zip, including data_utils.py, ml_experiments.py, app.py, research_evaluation.py and README.md. It also references the previously shared feature_extraction.py implementation and historical baseline class counts.

Module	Responsibilities
data_utils.py	Load MAT files, clean signals, derive candidate timestamp-reset groups
feature_extraction.py	Create complete non-overlapping 128-sample windows; compute 12 statistics
ml_experiments.py	Repeated random splits, tuning, classification metrics, macro ROC-AUC
research_evaluation.py	Candidate-group diagnostics, grouped cross-validation, exploratory permutations
app.py	Streamlit controls, visualizations, exports

Appendix B. Scientific cautions

The distinction between a candidate segment, a recording session, and a participant must remain explicit. Timestamps alone do not establish independence. Grouped results should be treated as exploratory until boundaries and recording provenance are verified.

This document intentionally separates historical reported counts, implemented code behavior, and illustrative diagrams from new measured experimental results, which are not yet available.